

Store Rating Platform Documentation

Overview

A full-stack web application that enables registered users to discover stores, submit ratings, and manage reviews through role-based access control.

This documentation covers:

- Project scope and implemented requirements
- Technology stack
- Backend and frontend architecture
- API endpoints
- User roles and permissions
- Setup and run instructions
- Database structure
- Security measures
- Known runtime behavior (localhost-only development)

Project Summary

Purpose

Build a centralized platform for store discovery and customer reviews with three user roles:

1. System Administrator
2. Normal User
3. Store Owner

The application supports:

- User registration and login
- Role-based dashboards and protected routes
- Store creation and management
- Rating submission and modification
- Password management
- Audit logging for admin actions

Key Completed Features

- JWT authentication and authorization
- Store search, filtering, sorting, and pagination
- 1-5 star rating submission and update
- Admin dashboards and analytics
- Store owner dashboard and review lists
- Password change functionality
- Full API documentation via Swagger
- Localhost-only frontend dev server configuration

Technology Stack

Layer	Technology
Frontend	React 18, Vite, Material-UI (MUI)
Frontend HTTP	Axios
Backend	Node.js, Express.js
Database	MySQL (via mysql2)
ORM	Sequelize
Authentication	JWT, bcryptjs
Validation	express-validator
API Docs	Swagger UI
Logging	Morgan

Requirements Implemented

System Administrator

- Add new stores, normal users, and admin users.
- Dashboard with: total users, total stores, total ratings.
- Create users with Name, Email, Password, Address, Role.
- View stores list with Name, Email, Address, Rating.
- View users list with Name, Email, Address, Role.
- Apply filters on listings.
- View user details, including Ratings for store owners.
- Logout support.

Normal User

- Sign up and login.
- Signup with Name, Email, Address, Password.
- Change password after login.
- View list of registered stores.
- Search stores by Name and Address.

- See store listings with Name, Address, Overall Rating, User's Submitted Rating.
- Submit and update ratings between 1 to 5 stars.
- Logout support.

Store Owner

- Login support.
- Change password after login.
- Dashboard with list of customers who rated the store.
- See average rating for owned store(s).
- Logout support.

Validation Rules

- Name: 20-60 characters.
- Address: max 400 characters.
- Password: 8-16 characters, at least one uppercase letter, and one special character.
- Email: standard email validation.

Project Structure

Backend

```
backend/  
  package.json  
  server.js  
  seed.js  
  src/  
    config/  
      db.config.js  
      swagger.js  
    controllers/  
      admin.controller.js  
      auth.controller.js  
      rating.controller.js  
      store.controller.js  
      storeOwner.controller.js  
    middleware/  
      auth.middleware.js  
      error.middleware.js  
    models/  
      index.js  
      user.model.js  
      store.model.js  
      rating.model.js  
      auditLog.model.js  
    repositories/  
      user.repository.js  
      store.repository.js  
      rating.repository.js  
      auditLog.repository.js  
    routes/  
      index.js  
      auth.routes.js  
      user.routes.js  
      admin.routes.js  
      store.routes.js  
      rating.routes.js  
      storeOwner.routes.js  
    services/  
      admin.service.js  
      auditLog.service.js  
      auth.service.js  
      rating.service.js  
      store.service.js  
      storeOwner.service.js  
    validators/  
      auth.validator.js  
      admin.validator.js  
      rating.validator.js  
      validationHelper.js  
    utils/  
      customErrors.js
```

Frontend

```
frontend/
  package.json
  vite.config.js
  src/
    App.jsx
    main.jsx
    theme.js
    components/
      Layout.jsx
      Navbar.jsx
      Sidebar.jsx
      StatCard.jsx
      StoreReviewsDialog.jsx
      ErrorBoundary.jsx
    context/
      AuthContext.jsx
      NotificationContext.jsx
      ThemeModeContext.jsx
    pages/
      Login.jsx
      Register.jsx
      Profile.jsx
    admin/
      Dashboard.jsx
      ReviewsManagement.jsx
      StoresManagement.jsx
      UsersManagement.jsx
      UserDetails.jsx
    user/
      Dashboard.jsx
    storeOwner/
      Dashboard.jsx
      RatingsList.jsx
    services/
      api.js
```

Backend Configuration

Environment Variables

Create `backend/.env` with these values:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_mysql_password
DB_NAME=store_rating_platform
JWT_SECRET=your_secret_key_min_32_characters
JWT_EXPIRES_IN=24h
NODE_ENV=development
CLIENT_URL=http://localhost:5173
DB_ALTER=false
```

Backend Scripts

- `npm start`: Start production backend
- `npm run dev`: Start backend with nodemon
- `npm run seed`: Run database seeder

Frontend Configuration

Environment Variables

Create `frontend/.env` with this value:

```
VITE_API_URL=http://localhost:5000/api
```

Frontend Scripts

- `npm run dev`: Start development frontend
- `npm run build`: Build production frontend
- `npm run preview`: Preview built frontend

Localhost-Only Development Behavior

The frontend dev server is configured to bind to `localhost` only.

- `frontend/vite.config.js` sets `server.host = 'localhost'`
- Development URL is `http://localhost:5173`
- The network URL display is disabled by binding to `localhost` exclusively

Backend API Endpoints

The backend exposes API routes under `/api` and Swagger docs at `/api-docs`.

Authentication

- `POST /api/auth/register`
- `POST /api/auth/login`
- `GET /api/auth/me`

User Profile

- `PUT /api/users/change-password`

Stores

- `GET /api/stores`
- `GET /api/stores/:storeId/reviews`

Ratings

- `POST /api/ratings`
- `PUT /api/ratings/:storeId`

Admin

- `GET /api/admin/dashboard`
- `POST /api/admin/users`
- `GET /api/admin/users`
- `GET /api/admin/users/:id`
- `PATCH /api/admin/users/:id/status`
- `DELETE /api/admin/users/:id`
- `POST /api/admin/stores`
- `GET /api/admin/stores`
- `PATCH /api/admin/stores/:id/status`
- `DELETE /api/admin/stores/:id`
- `GET /api/admin/ratings`
- `GET /api/admin/global-search`
- `GET /api/admin/audit-logs`

Store Owner

- `GET /api/store-owner/dashboard`
- `GET /api/store-owner/ratings`

Frontend Pages and Role-Based Navigation

Public Pages

- `/login`
- `/register`

Protected Pages

- `/` Home route redirects by role
- `/profile` Profile page for all authenticated users

Admin Pages

- `/admin/users`
- `/admin/stores`
- `/admin/users/:id`
- `/admin/reviews`

Store Owner Pages

- `/owner/ratings`

Normal User Pages

- `/` user dashboard with store list, rating modal, search, and sort

User Roles and Permissions

Role	Permissions
ADMIN	Full user/store management, analytics, audit logs, global search
STORE_OWNER	View owned stores, see average ratings, ratings history
USER	Search stores, submit and update ratings, profile management

Database Models

User

- `id`
- `name`
- `email`

- password
- address
- role (ADMIN, USER, STORE_OWNER)
- isActive

Store

- id
- name
- email
- address
- ownerId
- isActive

Rating

- id
- userId
- storeId
- rating (1-5)
- comment
- Unique user-store rating constraint

Audit Log

- id
- action
- performedBy
- targetId
- details
- createdAt

Security and Validation

Authentication

- JWT token authentication
- Tokens stored in `localStorage` on frontend
- Backend validates bearer tokens on protected routes

Authorization

- ADMIN middleware guard for admin routes
- STORE_OWNER guard for store owner routes
- USER guard for rating routes

Input Validation

- Registration and login validation
- Password complexity enforcement
- Rating limits between 1 and 5
- Optional comments with a max length

Secure Storage

- Passwords hashed with `bcryptjs`
- Sensitive configuration in `.env`
- CORS restricted to `http://localhost:5173`

Seed Data and Test Accounts

The backend contains a seeding script at `backend/seed.js` to populate initial data.

Example test accounts after seeding:

- Admin: `admin@platform.com / Password123!`
- Store Owner: `owner1@store.com / Password123!`
- User: `user1@consumer.com / Password123!`

How to Run the Project

Backend

```
cd backend
npm install
npm run seed
npm run dev
```

Backend URL: `http://localhost:5000`

Swagger docs: `http://localhost:5000/api-docs`

Frontend

```
cd frontend
npm install
npm run dev
```

Frontend URL: <http://localhost:5173>

Notes

- The frontend development server is intentionally configured to use only `localhost`.
 - The backend creates the database automatically if it does not exist.
 - API documentation is available through Swagger.
-

Additional Notes for Review

This document is built from the project implementation and reflects the current application structure, routes, and requirements.

If you want, I can also generate a `ProjectDocumentation.pdf` file from this markdown using a conversion tool once we confirm the required tooling is available.